

💡 Java me Variable Kitne Type ke Hote hain?

Java me mainly **3 types ke variables** hote hain:

Type	Kahan Use Hota Hai	Memory Lifetime
1. Local Variable	Method ke andar	Method khatam to memory free
2. Instance Variable	Class ke andar, lekin method ke bahar	Object ke saath survive karta hai
3. Static Variable	Class ke andar, <code>static</code> keyword ke saath	Class ke saath ek hi baar banta hai

✓ 1. Local Variable

★ Ye sirf method/function ke andar hi kaam karta hai.

🔒 Iska scope sirf us block/method tak hota hai.

```
class Test {  
    void show() {  
        int x = 10; // local variable  
        System.out.println("x = " + x);  
    }  
}
```

✓ 2. Instance Variable

★ Ye variable class ke andar hota hai (lekin kisi method ke bahar).

🔄 Har object ka apna alag instance hota hai.

```
class Student {  
    String name; // instance variable  
    int age;     // instance variable  
  
    void show() {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
}
```

✓ 3. Static Variable

★ `static` keyword ke saath use hota hai.

🌐 Ek hi copy poori class ke liye hoti hai (common for all objects).

```
class Student {  
    String name;  
    int age;  
    static String college = "ABC College"; // static variable  
  
    void show() {  
        System.out.println(name + " | " + age + " | " + college);  
    }  
}
```

```
}  
}
```

🔧 Situation: pass 100 students data hai

● 1. Bina Constructor ke kaise karega? (Manually har baar set karega)

```
Main.java  [Icons]  Run  Output  
1- class Student {  
2-     String name;  
3-     int age;  
4- }  
5-  
6- public class Main {  
7-     public static void main(String[] args) {  
8-         Student s1 = new Student();  
9-         s1.name = "Rahul";  
10-        s1.age = 20;  
11-  
12-        Student s2 = new Student();  
13-        s2.name = "Aman";  
14-        s2.age = 21;  
15-        System.out.println(s2.name);  
16-        System.out.println(s2.age);  
17-        // Socho agar 100 student honge to? 🤔🤔  
18-    }  
19- }  
20- }
```

Aman
21
=== Code Editor

➔ Constructor

```
Main.java  [Icons]  Run  Output  
1- class Student {  
2-     String name;  
3-     int age;  
4-  
5-     // Constructor: Object bante hi data set  
6-     Student(String name, int age) {  
7-         this.name = name;  
8-         this.age = age;  
9-     }  
10- }  
11-  
12- public class Main {  
13-     public static void main(String[] args) {  
14-         Student s1 = new Student("Rahul", 20);  
15-         Student s2 = new Student("Aman", 21);  
16-         System.out.println(s1.name);  
17-         System.out.println(s1.age);  
18-     }  
19- }  
20- }
```

Rahul
20
=== Code Editor

Bina constructor: Object banta hai... phir Ham manually data dete hai .

Constructor ke saath: Object bante hi data bhi set ho jaata hai ✔️ .

Main.java	Output
<pre>1- class Student { 2 String name; 3 4- Student(String name) { 5 this.name = name; // ✓ clear object ka name = parameter ka name 6 } 7- void display() { 8 System.out.println("Name: " + name); 9 } 10 } 11- public class Main { 12- public static void main(String[] args) { 13 Student s1 = new Student("Rahul"); 14 s1.display(); // Output: Name: Rahul ✓ 15 } 16 } 17</pre>	<pre>Name: Rahul === Code Execution</pre>

Defination :→

A constructor is a special method in Java that is used to initialize objects. It has the same name as the class and does **not have any return type**.

:::: →

constructor ek special function hota hai jo object banate hi automatic chal jaata hai, aur wo object ke variables ko set (initialize) karta hai. Iska naam class ke naam jaisa hota hai aur koi return type nahi hoti.

Toh Constructor Ki Zarurat Kyu Padti Hai?

1. Automatic Initialization — Object banate hi variables set karne ke liye

2. Code Saaf aur Maintain Karna Aasan Hotaa Hai

Jab tum bahut saare objects banao (jaise 100 students), toh har baar manually properties set karna boring aur error-prone ho sakta hai.

Constructor se:

```
Student s1 = new Student("Rahul");
Student s2 = new Student("Aman");
Student s3 = new Student("Priya");
```

Direct, fast, aur simple!

: initial Object

Without initialization: →

Main.java		Output
<pre> 1- class Student { 2- String name; 3- int age; 4- } 5 6- public class Main { 7- public static void main(String[] args) { 8- Student s1 = new Student(); 9- System.out.println(s1.name); // null 10- System.out.println(s1.age); // 0 11- } 12- } 13- </pre>		<pre> null 0 === Code Execution Success </pre>

With Constructor (Initialization):

Main.java		Output
<pre> 1- class Student { 2- String name; 3- int age; 4- // Constructor 5- Student(String n, int a) { 6- name = n; 7- age = a; 8- } 9- void showInfo() { 10- System.out.println("Name: " + name); 11- System.out.println("Age: " + age); 12- } 13- } 14- public class Main { 15- public static void main(String[] args) { 16- // Object banate waqt hi data initialize kar diya 17- Student s1 = new Student("Rahul", 20); 18- Student s2 = new Student("Aman", 22); 19- 20- s1.showInfo(); // Output: Name: Rahul, Age: 20 21- s2.showInfo(); // Output: Name: Aman, Age: 22 22- } 23- } 24- </pre>		<pre> Name: Rahul Age: 20 Name: Aman Age: 22 === Code Execution Success </pre>

→##

Main.java		Output
<pre> 1- class Student { 2- String name; 3- int age; 4- } 5 6- public class Main { 7- public static void main(String[] args) { 8- Student s1 = new Student(); 9- s1.name = "Rahul"; 10- s1.age = 20; 11- 12- System.out.println("Name: " + s1.name); 13- System.out.println("Age: " + s1.age); 14- } 15- } 16- </pre>		<pre> Name: Rahul Age: 0 === Code Execution Success </pre>

✖ Problem:

- Agar `s1.name` set karna bhool gaye to object **incomplete** rahega.
- Har object ke liye alag se `.name` aur `.age` set karna padta hai.

✓ 2. Constructor ke saath:

Main.java	Output
<pre> 1- class Student { 2- String name; 3- 4- Student(String name) { 5- this.name = name; // ✓ clear: object ka name = parameter ka 6- } 7- void display() { 8- System.out.println("Name: " + name); 9- } 10- } 11- public class Main { 12- public static void main(String[] args) { 13- Student s1 = new Student("Rahul"); 14- s1.display(); // Output: Name: Rahul ✓ 15- } 16- } 17- </pre>	Name: Rahul === Code Execution S...

→✓ — Java mein mainly 3 type constructor hote hain:

1. **Default Constructor** – bina argument ke
2. **Parameterized Constructor** – argument ke sath
3. **Copy Constructor** – ek object se dusra object banana

• 1: Bina kisi parameter ke hota hai

- Agar hum khud na banayein, to Java compiler automatic ek default constructor bana deta hai

Demo.java	Output
<pre> 1- class Demo { 2- Demo() { 3- System.out.println("Default Constructor"); 4- } 5- 6- public static void main(String[] args) { 7- Demo obj = new Demo(); // Constructor call 8- } 9- } 10- </pre>	Default Constructor === Code Execution S...

2. Parameterized Constructor

- **Isme parameters diye jaate hain** taaki object banate waqt values set ki jaa sakein.

Student.java	Output
<pre>1 class Student { 2 String name; 3 int age; 4 5 Student(String n, int a) { 6 name = n; 7 age = a; 8 } 9 10 void show() { 11 System.out.println("Name: " + name + ", Age: " + age); 12 } 13 14 public static void main(String[] args) { 15 Student s1 = new Student("Aman", 20); 16 s1.show(); 17 } 18 } 19</pre>	Name: Aman, Age: 20 === Code Execution Succ

◆ 3. Copy Constructor (Manual Banate Hai)

- **Ek object ki value dusre object me copy karta hai**
- Java me yeh automatic nahi hota, hume khud define karna padta hai.

Student.java	Output
<pre>1 class Student { 2 String name; 3 int age; 4 Student(String n, int a) { 5 name = n; 6 age = a; 7 } 8 // Copy Constructor 9 Student(Student s) { 10 name = s.name; 11 age = s.age; 12 } 13 void show() { 14 System.out.println("Name: " + name + ", Age: " + age); 15 } 16 17 public static void main(String[] args) { 18 Student s1 = new Student("Rahul", 22); 19 Student s2 = new Student(s1); // Copy constructor 20 s2.show(); 21 } 22 } 23</pre>	Name: Rahul, Age: 22 === Code Execution Successf

❓ *Copy Constructor Kya Hai?*

→ Copy constructor ka kaam hota hai:

→ Ek object ki **sabhi values** ko copy karke **naya object** banana.

⚙️ Example: Real Life Socho

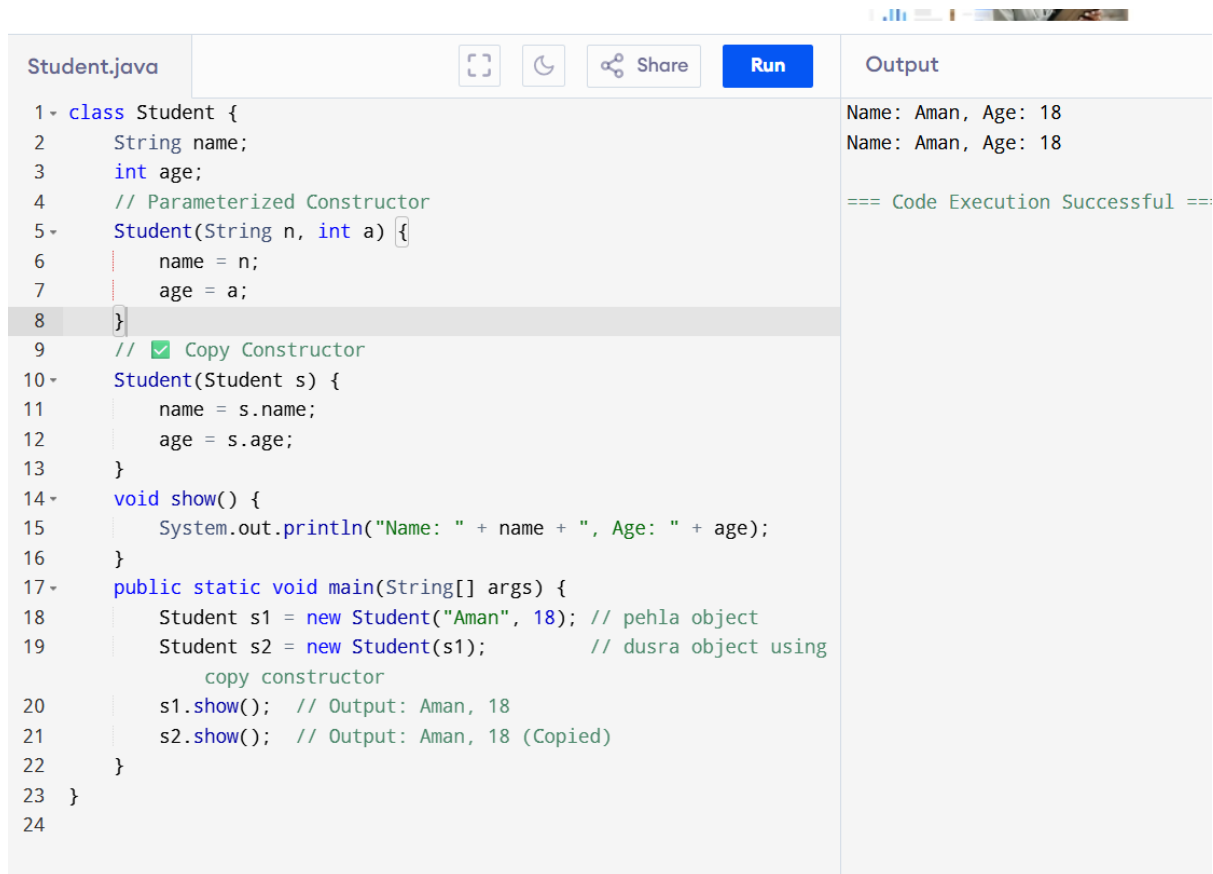
Tum ek "Student Form" bharte ho:

- Naam: *Aman*
- Age: *18*

Ab teacher kehta hai:

"Ek aur student ka form banao, same details ke saath."

Toh tum **copy** kar doge — **same name aur age**.



The screenshot shows a Java IDE with a file named `Student.java`. The code defines a `Student` class with two attributes: `String name` and `int age`. It includes a parameterized constructor, a copy constructor, and a `show()` method. The `main` method creates two `Student` objects: `s1` (with name "Aman" and age 18) and `s2` (created using the copy constructor from `s1`). Both objects call the `show()` method, resulting in the output "Name: Aman, Age: 18" and "Name: Aman, Age: 18 (Copied)". The IDE also shows a "Run" button and a "Share" icon. The output panel on the right displays the execution results.

```
1- class Student {
2-     String name;
3-     int age;
4-     // Parameterized Constructor
5-     Student(String n, int a) {
6-         name = n;
7-         age = a;
8-     }
9-     // ✅ Copy Constructor
10-    Student(Student s) {
11-        name = s.name;
12-        age = s.age;
13-    }
14-    void show() {
15-        System.out.println("Name: " + name + ", Age: " + age);
16-    }
17-    public static void main(String[] args) {
18-        Student s1 = new Student("Aman", 18); // pehla object
19-        Student s2 = new Student(s1);         // dusra object using
                                                // copy constructor
20-        s1.show(); // Output: Aman, 18
21-        s2.show(); // Output: Aman, 18 (Copied)
22-    }
23- }
24- }
```

Output

```
Name: Aman, Age: 18
Name: Aman, Age: 18

=== Code Execution Successful ===
```

→

Class and Object

*Class ki Definition:

Class ek blueprint ya template hoti hai jisme variables (data) aur methods (functions) define kiye jaate hain. Iske through hum real-world entities ko programming mein represent karte hain.

Simple shabdon mein:

Class ek design hota hai jisse hum ek object banate hain.

*Object ki Definition:

Object class ka ek real instance (actual copy) hota hai jo memory mein allocate hota hai. Object ke paas apni properties (data) hoti hain aur wo class ke methods ko use kar sakta hai.

Simple shabdon mein:

Object class ka ek bana hua example hai jo hum program mein use karte hain.

Main.java	Output
<pre>1 // Class ka example - Car class 2 class Car { 3 String brand; // Car ka brand (property) 4 int year; // Car ka manufacturing year (property) 5 6 public Car(String brand, int year) { 7 this.brand = brand; 8 this.year = year; 9 } 10 public void displayInfo() { 11 System.out.println("Brand: " + brand); 12 System.out.println("Year: " + year); 13 } 14 } 15 16 public class Main { 17 public static void main(String[] args) { 18 // Object banaya - Car class ka ek instance 19 Car myCar = new Car("Honda", 2019); 20 myCar.displayInfo(); 21 } 22 } 23</pre>	<pre>Brand: Honda Year: 2019 === Code Executing</pre>

Function aur Method: Definition & Difference

1. Function kya hai?

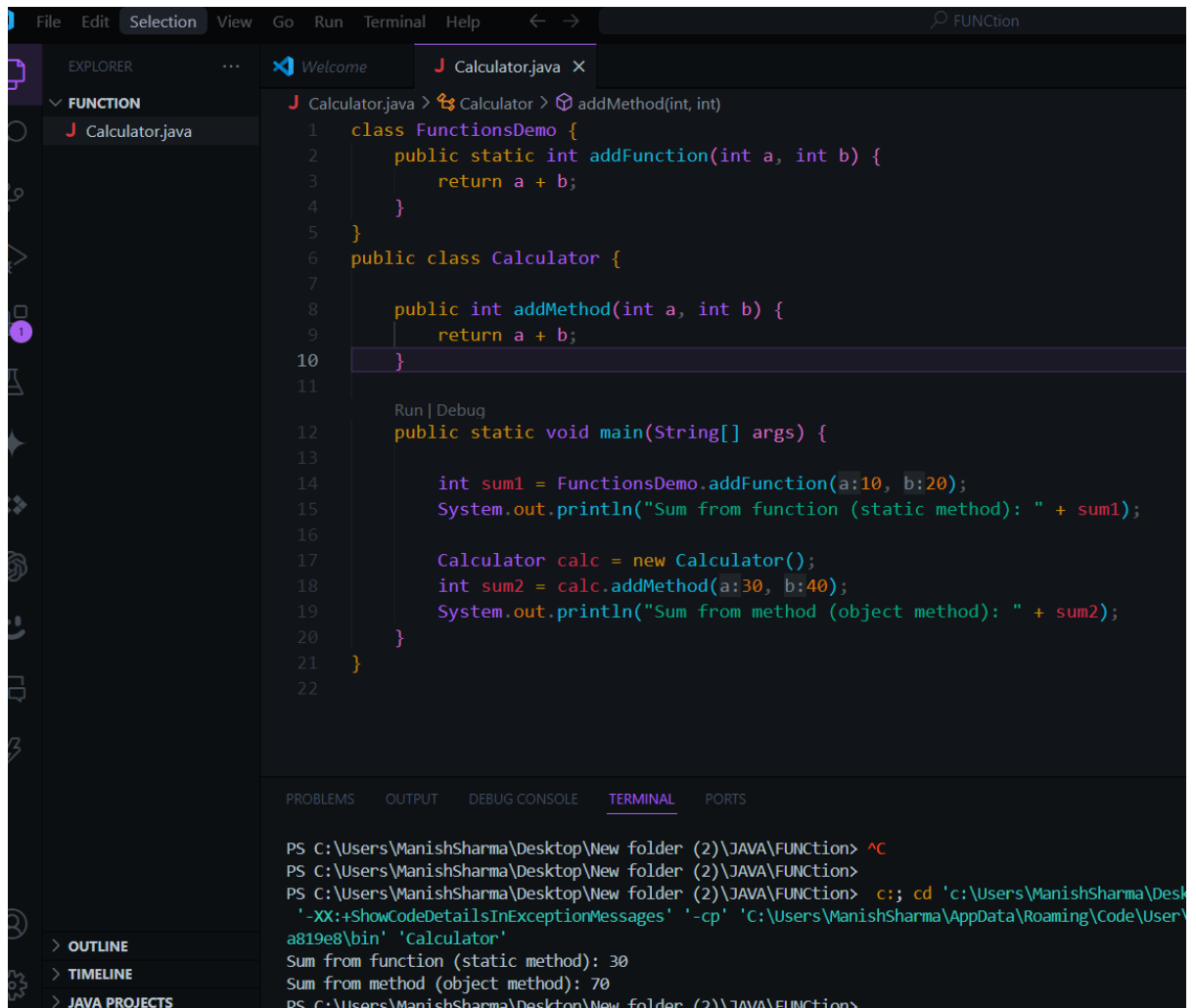
- **Function** ek aisa code block hota hai jo specific kaam karta hai aur optionally value return karta hai.
- General programming term hai, har language mein use hota hai (C, Python, Java, etc).
- Function independent ho sakta hai ya kisi class ka part nahi hota (jaise C ya Python mein).

2. Method kya hai?

- **Method** bhi ek function jaisa hi hota hai, lekin **ye sirf class ke andar hi hota hai**.
- Java mein, jo bhi function hota hai wo method hi hota hai kyunki Java mein sab kuch class ke andar hota hai.
- Matlab: Method = Class ke andar defined function.

Term	Definition
Function	Independent code block jo task karta hai, value return kar sakta hai.
Method	Class ke andar defined function, jo object ke saath related hota hai.

Aspect	Function	Method
Location	Class ke bahar ya andar ho sakta hai (general)	Sirf class ke andar hota hai
Object-oriented term	General programming term	Object-oriented programming ka part
Usage in Java	Java mein method ke alawa function nahi hota	Java mein functions ko methods kehte hain
Call karne ka tarika	Direct call ho sakta hai	Object ke through ya class ke andar call hota hai



```
File Edit Selection View Go Run Terminal Help
EXPLORER
FUNCTION
Calculator.java
Calculator.java
class FunctionsDemo {
    public static int addFunction(int a, int b) {
        return a + b;
    }
}
public class Calculator {
    public int addMethod(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        int sum1 = FunctionsDemo.addFunction(a:10, b:20);
        System.out.println("Sum from function (static method): " + sum1);

        Calculator calc = new Calculator();
        int sum2 = calc.addMethod(a:30, b:40);
        System.out.println("Sum from method (object method): " + sum2);
    }
}

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ManishSharma\Desktop\New folder (2)\JAVA\FUNCTION> ^C
PS C:\Users\ManishSharma\Desktop\New folder (2)\JAVA\FUNCTION>
PS C:\Users\ManishSharma\Desktop\New folder (2)\JAVA\FUNCTION> c:; cd 'c:\Users\ManishSharma\Desktop\New folder (2)\JAVA\FUNCTION'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ManishSharma\AppData\Roaming\Code\User\
a819e8\bin' 'Calculator'
Sum from function (static method): 30
Sum from method (object method): 70
PS C:\Users\ManishSharma\Desktop\New folder (2)\JAVA\FUNCTION>
```

1. Predefined Parameters (Java mein direct nahi hote)

- **Predefined parameters** wo hote jo kisi programming language ya framework khud provide karta hai, jise user explicitly define nahi karta.
- Java mein directly predefined parameters nahi hote, lekin kuch special cases hain jaise `main(String[] args)` mein `args` parameter system se predefined hota hai (command line arguments ke liye).

```
public static void main(String[] args) {
    // yaha args predefined parameter hai jisme command line arguments aate hain
}
```

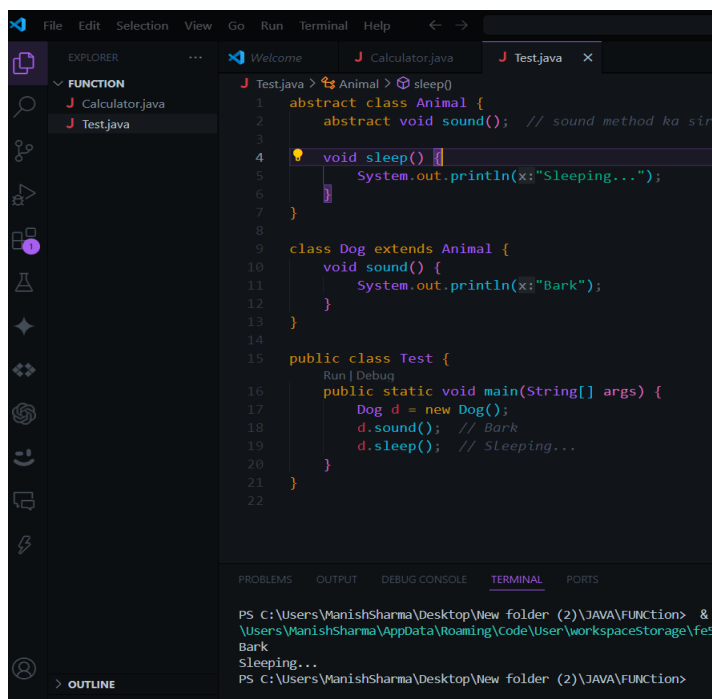
2. User-Defined Parameters

- **User-defined parameters** wo hote hain jo programmer khud method/function ke andar define karta hai.
- Ye parameters method ko input dete hain taaki method unke basis pe kaam kar sake.

```
public int add(int a, int b) {  
    return a + b;  
}
```

1. Abstract (सपना जैसा - पूरा नहीं, bas idea deta hai)

- **Abstract class:** Aisi class jo complete nahi hoti, sirf idea deti hai.
- **Abstract method:** Aisa method jiska body nahi hota, bas naam hota hai, baad mein subclass uska pura meaning deta hai.



**** Jab bhi koi method abstract hota hai, toh us class ko bhi abstract banana padta hai.**

Main.java

Share

Run

```

1 // Step 1: Abstract class banaya
2- abstract class Animal {
3     // Abstract method – sirf signature, body nahi
4     abstract void sound();
5- void sleep() {
6         System.out.println("Animal is sleeping 🐾");
7     }
8- class Dog extends Animal {
9-     void sound() {
10        System.out.println("Dog barks 🐶");
11    }
12 }
13- class Cat extends Dog {
14-     void sound() {
15        System.out.println("CAT Meow");
16    }
17- public class Main {
18-     public static void main(String[] args) {
19         Dog d = new Dog();
20         Cat d1 = new Cat();
21         d1.sound(); // Output: Dog barks 🐶
22         d.sound(); // Output: Dog barks 🐶
23         d.sleep(); // Output: Animal is sleeping 🐾
24     }
25 }
26

```

Output

```

CAT Me
Dog barks ?
Animal is sleeping ?

=== Code Execution Suc

```



🔒 final Keyword in Java

📖 Definition:

Java me `final` keyword ka use kisi **cheez ko fix/constant** banane ke liye hota hai — taaki usme **koi change na ho sake**.

💡 `final` 3 cheezon ke saath use hota hai:

Use Case	Meaning
<code>final variable</code>	Value fix ho gayi – badal nahi sakte (constant)
<code>final method</code>	Method override nahi ho sakta subclass me
<code>final class</code>	Class ko inherit (extend) nahi kar sakte

💡 1. Final Variable

Main.java

Share

Run

```

1 final int x = 10;
2 x = 20; // ❌ Error: can't change final variable
3

```

- `x` ki value **fix** ho gayi.

- Ab isse change karne ki koshish karoge toh **error aayega**.

2. Final Method

```
A.java
1- class A {
2-     final void display() {
3-         System.out.println("Hello from A");
4-     }
5- }
6-
7- class B extends A {
8-     // ❌ Error: Cannot override final method
9-     // void display() { System.out.println("Hello from B"); }
10- }
11- |
```

CODE :➡

Main.java	Output
<pre>1- class A { 2- final void display() { 3- System.out.println("Hello from A"); 4- } 5- } 6- 7- class B extends A { 8- // ❌ Error: Cannot override final method from class A 9- void display() { 10- System.out.println("Hello from B"); 11- } 12- } 13- 14- public class Main { 15- public static void main(String[] args) { 16- B obj = new B(); 17- obj.display(); 18- } 19- } 20-</pre>	<pre>ERROR! Main.java:9: error: display() in B cannot override displ void display() { ^ overridden method is final 1 error === Code Exited With Errors ===</pre>

→ERROR CODE:

Main.java	Output
<pre>1- class Parent { 2 // Final method 3- public final void showMessage() { 4 System.out.println("Hello from Parent class (final method)"); 5 } 6 } 7- class Child extends Parent { 8 // Attempting to override this method will give an error 9- public void showMessage() { 10 System.out.println("Trying to override"); // ❌ Not allowed 11 } 12 } 13 14- public class Main { 15- public static void main(String[] args) { 16 Child obj = new Child(); 17 obj.showMessage(); // This will call the Parent class's final 18 method 19 } 20 }</pre>	ERROR! Main.java:9: error: showMessage() in Child cannot override Parent public void showMessage() { ^ overridden method is final 1 error === Code Exited With Errors ===

Right Code →

→

Main.java	Output
<pre>1- class Parent { 2 // Final method 3- public final void showMessage() { 4 System.out.println("Hello from Parent class (final method)"); 5 } 6 } 7- class Child extends Parent { 8 // Attempting to override this method will give an error 9- // public void showMessage() { 10 // System.out.println("Trying to override"); // ❌ Not allowed 11 // } 12 } 13 14- public class Main { 15- public static void main(String[] args) { 16 Child obj = new Child(); 17 obj.showMessage(); // This will call the Parent class's final 18 method 19 } 20 }</pre>	Hello from Parent class (final method) === Code Execution Successful ===

→

✓ Final Class in Java

Jab kisi class ko `final` declare kiya jaata hai, to **us class ko inherit (extend) nahi kiya ja sakta.**

Yaani, koi doosri class us class ko **subclass nahi bana sakti.**

🔒 Kyon use karte hain?

- Jab aap chahte hain ki **koi class ka structure badla na jaaye**, tab use `final` banaya jaata hai.
- Yeh **security aur stability** ke liye hota hai.

Main.java	Output
<pre>1 final class Animal { 2 public void sound() { 3 System.out.println("Animal makes a sound"); 4 } 5 } 6 // This will cause a compile-time error 7 // class Dog extends Animal { 8 // public void sound() { 9 // System.out.println("Dog barks"); 10 // } 11 // } 12 public class Main { 13 public static void main(String[] args) { 14 Animal a = new Animal(); 15 a.sound(); 16 } 17 } 18</pre>	<pre>Animal makes a sound === Code Execution Successful ===</pre>

Abstract Method: ➡

"An abstract method is a method that has no body and must be implemented by the subclass.

Simple Explanation (Hinglish Style):

Abstract method ek **aesa method hota hai jisme koi kaam (code) nahi hota**, sirf ye bataya jaata hai ki **ye method har child class me hona chahiye**.

Ye sirf **abstract class** ke andar hota hai.

```
Main.java
1 // Abstract class banaya
2- abstract class Animal {
3     // Abstract method - body nahi di
4     abstract void sound();
5- void sleep() {
6     System.out.println("Sleeping...");
7 }
8 }
9- class Dog extends Animal {
10- void sound() {
11     System.out.println("Dog barks"); }
12 }
13- class Cat extends Dog{
14- void sound(){
15     System.out.println("meow meow");
16 }
17 }
18- public class Main {
19- public static void main(String[] args) {
20     Dog d = new Dog();
21     Cat d1 = new Cat();
22     d.sound(); // Output: Dog barks
23     d1.sound(); // Output: Dog barks
24     d.sleep(); // Output: Sleeping...
25 }
26 }
```

Output

Dog barks
meow meow
Sleeping...
=== Code Execution

Definition of Interface: ➡

"Interface is a rule book that tells what a class must do, but not how."

EX :=>

"Interface ek **contract** ya **rule book** jaisa hota hai.

Jaise school ke teacher bolte hain —

'Har student ko homework karna hai, test dena hai' —

to woh sirf **rules batate hain**, kaise karna hai ye student pe chhod dete hain."

SIMPLE ➡

(Interface ka matlab hai — kaam karne ka promise, kaam kaise karna hai woh class batayegi.")

Main.java

Share

Run

Output

```

1 // Interface declaration
2 interface Animal {
3     void sound(); // abstract method (no body)
4     void sleep();
5 }
6 // Class implementing the interface
7 class Dog implements Animal {
8     public void sound() {
9         System.out.println("Dog barks");
10    }
11
12    public void sleep() {
13        System.out.println("Dog sleeps");
14    }
15 }
16
17 public class Main {
18     public static void main(String[] args) {
19         Dog d = new Dog();
20         d.sound();
21         d.sleep();
22     }
23 }
24

```

Dog barks
Dog sleeps
=== Code Exe

DIFFERENCE :

🔍 Feature	❏ Abstraction (Abstract Class)	📄 Interface
✈ Matlab kya hai?	Base class jisme kuch method ka kaam chhupa hota hai	Sirf rules ya method names define karta hai
🔧 Code hota hai?	Abstract + normal method dono ho sakte hain	Sirf method names hote hain, no code
⚙ Java me kaise?	<code>abstract class</code> se hota hai	<code>interface</code> keyword se banta hai
❏ Method body allowed?	Haan, kuch method ka code ho sakta hai	Nahi, sab method without body hote hain
❏ Variables	Can have variables (with different types)	Variables are public + static + final only
⇄ Inheritance	<code>extends</code> se hota hai (ek hi class extend hoti hai)	<code>implements</code> se hota hai (multiple interface allowed)
❏ Kab use kare?	Jab kuch code common ho aur kuch chhupana ho	Jab sirf rules dene ho aur 100% abstraction chahiye

ABSTRACT :

Main.java	Output
<pre>1- abstract class Vehicle { 2- abstract void start(); // abstract method, no body 3- 4- void stop() { 5- System.out.println("Vehicle stopped."); 6- } 7- } 8- 9- class Bike extends Vehicle { 10- void start() { 11- System.out.println("Bike started."); 12- } 13- } 14- 15- public class Main { 16- public static void main(String[] args) { 17- Bike b = new Bike(); 18- b.start(); // Output: Bike started. 19- b.stop(); // Output: Vehicle stopped. 20- } 21- } 22-</pre>	<pre>Bike started. Vehicle stopped. === Code Execution Successful ===</pre>

Interface :

Main.java	Output
<pre>1- interface Printer { 2- void print(); // abstract method, no body 3- } 4- 5- class InkjetPrinter implements Printer { 6- public void print() { 7- System.out.println("Printing using Inkjet Printer."); 8- } 9- } 10- 11- public class Main { 12- public static void main(String[] args) { 13- InkjetPrinter ip = new InkjetPrinter(); 14- ip.print(); // Output: Printing using Inkjet Printer. 15- } 16- } 17-</pre>	<pre>Printing using Inkjet Printer. === Code Execution Successful ===</pre>

1. Interface ka object banta hai?

Nahi !

Interface ka **direct object nahi bana sakte** Java me.

Kyoon ?

Interface sirf ek **blueprint (rules ka set)** hota hai, usme method ke bodies nahi hote. Isliye **uska koi asli object nahi hota.**

2. Abstract class ka object banta hai?

Nahi!

Abstract class ka bhi **direct object nahi bana sakte.**

Kyoon?

Kyunki abstract class me **kuch methods ka code nahi hota** (abstract methods), to Java ko pata nahi hota ki wo pura kaam kaise karega.

Par dono ka object kaise banta hai?

- **Interface:**
Usko implement karne wali **class ka object** banta hai, jo saare methods ka code deta hai.
- **Abstract class:**
Usko extend karne wali **child class ka object** banta hai, jo abstract methods ko implement karta hai.

DIFFERENCE CODE ➔ -----

Main.java	Output
<pre>1- abstract class Animal { 2- void breathe() { 3- System.out.println("Animal is breathing"); 4- } 5- 6- abstract void sound(); 7- } 8- class Dog extends Animal { 9- void sound() { 10 System.out.println("Dog barks"); 11 } 12 } 13- public class Main { 14- public static void main(String[] args) { 15 Dog d = new Dog(); 16 d.breathe(); // Common code use kiya 17 d.sound(); // Specific implementation diya 18 } 19 } 20</pre>	<pre>Animal is breathing Dog barks === Code Execution Successful</pre>

Main.java	Output
<pre>1- interface Flyable { 2- void fly(); 3- } 4- interface Swimmable { 5- void swim(); 6- } 7- class Duck implements Flyable, Swimmable { 8- public void fly() { 9- System.out.println("Duck is flying"); 10- } 11- 12- public void swim() { 13- System.out.println("Duck is swimming"); 14- } 15- } 16- 17- public class Main { 18- public static void main(String[] args) { 19- Duck d = new Duck(); 20- d.fly(); // Rule follow kiya 21- d.swim(); // Rule follow kiya 22- } 23- } 24-</pre>	<pre>Duck is flying Duck is swimming === Code Execution</pre>

- Duck ek saath **do alag-alag abilities** (fly aur swim) le sakta hai.
- Java class multiple abstract class extend nahi kar sakti, par multiple interface implement kar sakti hai.

LAMBDA FUNCTION : ➔

("Lambda expression ek chhota function hota hai jo ek method ko inline likhne ka simple tareeka deta hai.")

➔

"Interface ka direct object nahi banta, par usko use karne ke 2 tareeke hote hain — Lambda ya Class."

➔

w/o LAMBDA ==>

Main.java	Output
<pre> 1 // Step 1: Interface banana 2 interface Calculator { 3 int add(int a, int b); 4 } 5 class MyCalculator implements Calculator { 6 7 public int add(int a, int b) { 8 return a + b; 9 } 10 } 11 public class Main { 12 public static void main(String[] args) { 13 MyCalculator c = new MyCalculator(); 14 int result = c.add(5, 3); 15 // Result print kiya 16 System.out.println("Sum is: " + result); 17 } 18 } 19 </pre>	<pre> Sum is: 8 === Code Exec </pre>

WITH LAMBDA →

Main.java	Output
<pre> 1 interface Calculator { 2 int add(int a, int b); 3 } 4 5 public class Main { 6 public static void main(String[] args) { 7 // Calculator c = new Calculator(); // ERROR: interface ka object nahi banta 8 9 // Lambda se Calculator interface ka object ban raha hai 10 Calculator c = (a, b) -> a + b; 11 12 System.out.println(c.add(5, 3)); // Output: 8 13 } 14 } 15 </pre>	<pre> 8 === Code </pre>

→

❓ Inheritance in Java

✓ Definition (Simple Words):

Inheritance ka matlab hota hai ek class (child) dusri class (parent) ke properties aur methods ko use kar sakti hai.

Jaise ek "Bachha apne Maa-Baap ki qualities inherit karta hai", waise hi programming mein ek class dusri class se features "inherits" karti hai.

Type	Description
1. Single Inheritance	Ek class dusri ek class se inherit karti hai
2. Multilevel Inheritance	Ek class dusri se, wo kisi aur se (chain)
3. Hierarchical Inheritance	Ek parent se kai child classes inherit
4. Multiple Inheritance (via Interface)	Java class multiple interfaces inherit karti hai

```

1  class Animal12 { 1 usage 1 inheritor
2      void eat() { 1 usage
3          System.out.println("Animal eats food");
4      }
5  }
6
7  class Dog2 extends Animal12 { 2 usages
8      void bark() { 1 usage
9          System.out.println("Dog barks");
10     }
11 }
12
13 public class Test_1 {
14     public static void main(String[] args) {
15         Dog2 d = new Dog2();
16         d.eat(); // Animal class ka method
17         d.bark(); // Dog class ka method
18     }
19 }
20

```

Run Test_1

```

C:\Users\ManishSharma\.jdk\corretto-23.0.2\bin\java
Animal eats food
Dog barks
Process finished with exit code 0

```

Types of Inheritance in Java ->

 What is Inheritance?

Inheritance ek object-oriented concept hai jisme ek class (child/subclass) dusri class (parent/superclass) ke properties (variables) aur methods (functions) ko use kar sakti hai. Isse **code reuse** hota hai, aur naye features add karna aasaan ho jata hai.

12 Types of Inheritance in Java:

1 Single Inheritance

✓ Java Supported

☞ Isme **ek class**, dusri **ek hi class** se inherit karti hai.
Ek **child class**, ek **parent class** ke methods/properties use karti hai.

```
class A { }  
class B extends A { } // B inherits A
```

2 Multilevel Inheritance

✓ Java Supported

☞ Ek class **dusri class se inherit** karti hai, aur fir teesri class **usse inherit** karti hai.
Yani **chain of inheritance** banti hai.

```
class A { }  
class B extends A { }  
class C extends B { }
```

3 Hierarchical Inheritance

✓ Java Supported

☞ **Ek parent class se multiple child classes** inherit karti hain.
Sabhi child classes **same parent ke method** ko access kar sakti hain

```
class A { }  
class B extends A { }  
class C extends A { }
```

Inheritance Type	Supported in Java	Description
Single Inheritance	✓ Yes	One child class inherits one parent class
Multilevel Inheritance	✓ Yes	Inheritance in chain ($A \rightarrow B \rightarrow C$)
Hierarchical Inheritance	✓ Yes	One parent class, many child classes

Inheritance Type	Supported in Java	Description
Multiple Inheritance	✗ No (class)	Allowed via interface only
Hybrid Inheritance	✗ No (class)	Achieved via interface combination

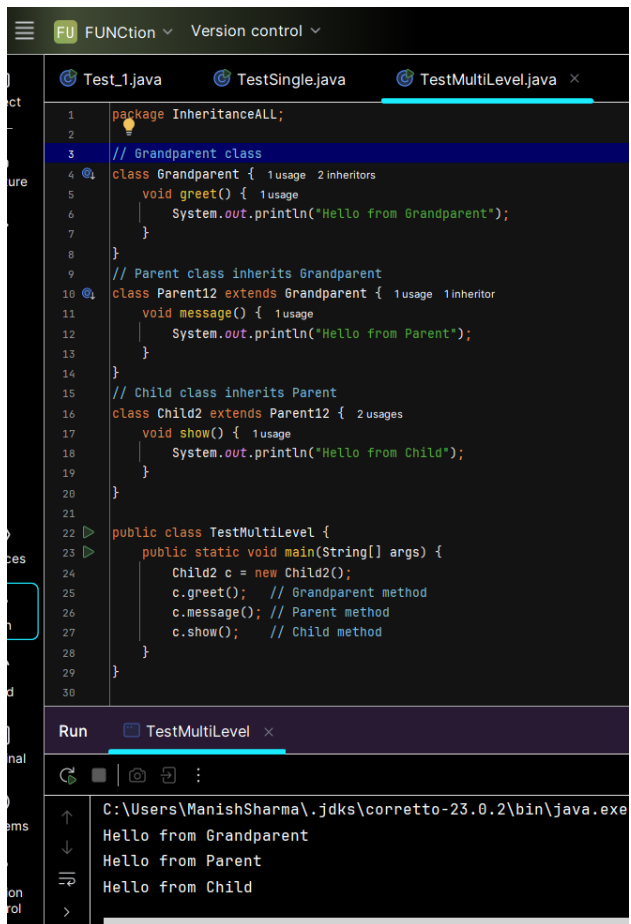
CODE :: →

```

1 package InheritanceALL;
2
3 // Parent class
4 class Parent { 1 usage 1 inheritor
5     void show() { 1 usage
6         System.out.println("This is parent class");
7     }
8 }
9
10 // Child class inherits Parent
11 class Child extends Parent { 2 usages
12     void display() { 1 usage
13         System.out.println("This is child class");
14     }
15 }
16
17 public class TestSingle {
18     public static void main(String[] args) {
19         Child c = new Child();
20         c.show(); // Parent class method
21         c.display(); // Child class method
22     }
23 }
24
Run TestSingle
C:\Users\ManishSharma\.jdk\corretto-23.0.2\bin\java.
This is parent class
This is child class

```


➔MULTILEVEL ➔



```
1 package InheritanceALL;
2
3 // Grandparent class
4 class Grandparent { 1 usage 2 inheritors
5     void greet() { 1 usage
6         System.out.println("Hello from Grandparent");
7     }
8 }
9 // Parent class inherits Grandparent
10 class Parent12 extends Grandparent { 1 usage 1 inheritor
11     void message() { 1 usage
12         System.out.println("Hello from Parent");
13     }
14 }
15 // Child class inherits Parent
16 class Child2 extends Parent12 { 2 usages
17     void show() { 1 usage
18         System.out.println("Hello from Child");
19     }
20 }
21
22 public class TestMultiLevel {
23     public static void main(String[] args) {
24         Child2 c = new Child2();
25         c.greet(); // Grandparent method
26         c.message(); // Parent method
27         c.show(); // Child method
28     }
29 }
30
```

Run TestMultiLevel

C:\Users\ManishSharma\.jdk\corretto-23.0.2\bin\java.exe
Hello from Grandparent
Hello from Parent
Hello from Child

3 Hierarchical Inheritance

The screenshot shows an IDE with the following components:

- Top Bar:** Displays 'FU FUNCTION' and 'Version control'.
- Left Sidebar:** Contains icons for Project, Structure, Services, Run, Build, Terminal, Problems, and Version Control.
- Editor:** Shows the code for `TestHierarchical.java`. The code defines a base class `Animal` with an `eat()` method, and two subclasses `Dog` and `Cat` that override `bark()` and `meow()` respectively. A `TestHierarchical` class contains a `main` method that creates instances of `Dog` and `Cat` and calls their methods.
- Run Console:** Shows the output of the program: `Animal is eating`, `Dog barks`, `Animal is eating`, and `Cat meows`.

```
1 package InheritanceALL;
2 // Parent class
3 class Animal { 2 usages 2 inheritors
4     void eat() { 2 usages
5         System.out.println("Animal is eating");
6     }
7 }
8 // Child class 1
9 class Dog extends Animal { 2 usages
10     void bark() { 1 usage
11         System.out.println("Dog barks");
12     }
13 }
14 // Child class 2
15 class Cat extends Animal { 2 usages
16     void meow() { 1 usage
17         System.out.println("Cat meows");
18     }
19 }
20 public class TestHierarchical {
21     public static void main(String[] args) {
22         Dog d = new Dog();
23         d.eat(); // From Animal class
24         d.bark(); // Dog class
25
26         Cat c = new Cat();
27         c.eat(); // From Animal class
28         c.meow(); // Cat class
29     }
30 }
```

Run TestHierarchical

Animal is eating
Dog barks
Animal is eating
Cat meows

4 Multiple Inheritance (Using Interfaces)

```
1 package InheritanceALL;
2
3 interface Printable { 1 usage 1 implementation
4     void print(); 1 usage 1 implementation
5 }
6 interface Showable { 1 usage 1 implementation
7     void show(); 1 usage 1 implementation
8 }
9 class Demo implements Printable, Showable { 2 usages
10     public void print() { 1 usage
11         System.out.println("Print method");
12     }
13     public void show() { 1 usage
14         System.out.println("Show method");
15     }
16 }
17
18 public class TestMultiple {
19     public static void main(String[] args) {
20         Demo obj = new Demo();
21         obj.print();
22         obj.show();
23     }
24 }
25
```

Run TestMultiple x

Print method
Show method
Process finished with exit code 0

5 Hybrid Inheritance (Using Interfaces + Classes)

```
1 package InheritanceALL;
2 interface A { 1 usage 2 implementations
3     void methodA(); 1 usage 1 implementation
4 }
5 interface B { 1 usage 2 implementations
6     void methodB(); 1 usage 1 implementation
7 }
8 // Class C implements two interfaces
9 class C implements A, B { 1 usage 1 inheritor
10     public void methodA() { 1 usage
11         System.out.println("Method A");
12     }
13     public void methodB() { 1 usage
14         System.out.println("Method B");
15     }
16 }
17 // Class D inherits class C and interface A again (just example)
18 class D extends C { 2 usages
19     void methodD() { 1 usage
20         System.out.println("Method D");
21     }
22 }
23 public class TestHybrid {
24     public static void main(String[] args) {
25         D obj = new D();
26         obj.methodA();
27         obj.methodB();
28         obj.methodD();
29     }
30 }
31
32
```

Run TestHybrid

C:\Users\ManishSharma\.jdk\corretto-23.0.2\bin\java.exe -cp C:\Users\ManishSharma\.jdk\corretto-23.0.2\bin TestHybrid

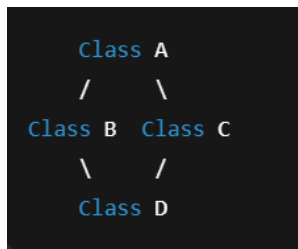
Method A
Method B
Method D

❓ Why Multiple Inheritance with Interfaces only?

1. Java mein Multiple Inheritance (Class ke through) allowed nahi kyun?

- Agar ek class **do ya zyada classes** se directly inherit kare (multiple inheritance), to "**Diamond Problem**" create ho sakta hai.

🔍 Diamond Problem kya hota hai?



2. Interface kaise solve karta hai?

- Interfaces mein **sirf method declarations** hote hain (method body nahi hoti — ya phir Java 8 se default methods hote hain, jo alag handle hote hain).
- Isliye **ambiguity kam ho jati hai**, kyunki class ko clearly methods implement karne hote hain.
- Interfaces multiple ho sakte hain, kyunki koi method body nahi hoti ya developer khud clearly define karta hai.

Point	Class Inheritance	Interface Inheritance
Multiple inheritance	Not allowed	Allowed
Diamond problem	Ho sakta hai (ambiguity)	Nahi hota (method body nahi)
Method implementation	Parent class se directly milti	Implement karni hoti hai child class ko
Flexibility	Limited	Zyada flexibility deta hai

Encapsulation

is a concept in object-oriented programming where we **wrap the data (variables)** and **code (methods)** together into a single unit (class), and **restrict direct access** to some of the object's components by making them `private` and accessing them through `getter` and `setter` methods.

"**Encapsulation** ka matlab hota hai — data ko **private** rakhna, aur **getter-setter** ke through access dena."

Main.java	Output
<pre> 1- class User { 2 private String password; 3 } 4 5- public class Main { 6 public static void main(String[] args) { 7 User u = new User(); 8 9 // User ne password set kiya 10 u.password = "12345"; 11 // Hacker ne directly password change kar diya 🤖 12 // u.password = "hacked123"; 13 14 System.out.println("Password is: " + u.password); 15 } 16 } 17 </pre>	<pre> ERROR! Main.java:10: error: password has private access in User u.password = "12345"; ^ ERROR! Main.java:13: error: password has private access in User u.password = "hacked123"; ^ ERROR! Main.java:15: error: password has private access in User System.out.println("Password is: " + u.password); ^ 3 errors === Code Exited With Errors === </pre>

Solution ➡:

Main.java	Output
<pre> 1- class User { 2 private String password; 3 public void setPassword(String pwd) { 4 password = pwd; 5 } 6 // Getter method - password read karne ke liye 7 public String getPassword() { 8 return password; 9 } 10 } 11- public class Main { 12 public static void main(String[] args) { 13 User u = new User(); 14 15 u.setPassword("12345"); 16 System.out.println("Password is: " + u.getPassword()); 17 } 18 } 19 20 </pre>	<pre> Password is: 12345 === Code Execution Su </pre>

🔄 Polymorphism – Definition

Polymorphism means "many forms". In Java, it allows **one function, method, or object to behave differently in different situations**.

📖 Simple Definition (Easy Words):

Jab **ek hi method ya object** ka **multiple forms me use** hota hai — usko **Polymorphism** kehte hain.

Type	How it happens	Also called as
1. Compile-time Method Overloading	Static Polymorphism	
2. Run-time	Method Overriding	Dynamic Polymorphism

◆ 1. Method Overloading (Compile-Time)

Same method name, **different parameters** — compiler decides which one to run.

Main.java	   Share	Run	Output
<pre>1- class Calculator { 2- void add(int a, int b) { 3- System.out.println("Sum: " + (a + b)); 4- } 5- 6- void add(int a, int b, int c) { 7- System.out.println("Sum: " + (a + b + c)); 8- } 9- void add(int a, int b, int c, int d) { 10- System.out.println("Sum: " + (a + b + c + d)); 11- } 12- } 13- public class Main { 14- public static void main(String[] args) { 15- Calculator c = new Calculator(); 16- c.add(2, 3); // calls 2-arg version 17- c.add(1, 2, 3); // calls 3-arg version 18- c.add(1, 2, 3, 5); // calls 4-arg version 19- } 20- } 21 22 23</pre>			<pre>Sum: 5 Sum: 6 Sum: 11 === Code Execution</pre>

◆ 2. Method Overriding (Run-Time)

Same method name and parameters — **but in child class**, different behavior.

Main.java	Output
<pre>1 class Animal { 2 void sound() { 3 System.out.println("Animal makes sound"); 4 } 5 } 6 7 class Dog extends Animal { 8 void sound() { 9 System.out.println("Dog barks"); 10 } 11 } 12 public class Main { 13 public static void main(String[] args) { 14 Animal a = new Dog(); // Runtime polymorphism 15 a.sound(); // Output: Dog barks 16 } 17 } 18 19 20</pre>	Dog barks === Code Execution

Abstraction – Definition

Abstraction means **hiding the internal implementation** and **showing only the essential details** to the user.

Main.java	Output
<pre> 1 // Abstract Class 2 abstract class Animal { 3 abstract void sound(); 4 // Normal method 5 void eat() { 6 System.out.println("Animal is eating"); 7 } 8 } 9 // Subclass 10 class Cat extends Animal { 11 // Implement abstract method 12 void sound() { 13 System.out.println("Cat says Meow"); 14 } 15 } 16 // Main class 17 public class Main { 18 public static void main(String[] args) { 19 Cat c = new Cat(); 20 c.sound(); // Output: Cat says Meow 21 c.eat(); // Output: Animal is eating 22 } 23 } 24 </pre>	<pre> Cat says Meow Animal is eating === Code Execution Successful === </pre>



Interface

Main.java	Output
<pre> 1 // Interface 2 interface Vehicle { 3 void start(); // abstract method by default 4 void stop(); // abstract method by default 5 } 6 // Class implementing interface 7 class Car implements Vehicle { 8 public void start() { 9 System.out.println("Car starts with push button"); 10 } 11 12 public void stop() { 13 System.out.println("Car stops with brakes"); 14 } 15 } 16 // Main class 17 public class Main { 18 public static void main(String[] args) { 19 Car c = new Car(); 20 c.start(); // Output: Car starts with push button 21 c.stop(); // Output: Car stops with brakes 22 } 23 } 24 </pre>	<pre> Car starts with push button Car stops with brakes === Code Execution Successful === </pre>

Abstract Class

abstract class Animal {}

Can have normal + abstract methods

Interface

interface Vehicle {}

Only abstract methods (Java 8+ allows default too)

Abstract Class

Use `extends` to inherit

One class can extend 1 abstract class

Interface

Use `implements` to inherit

One class can implement multiple interfaces